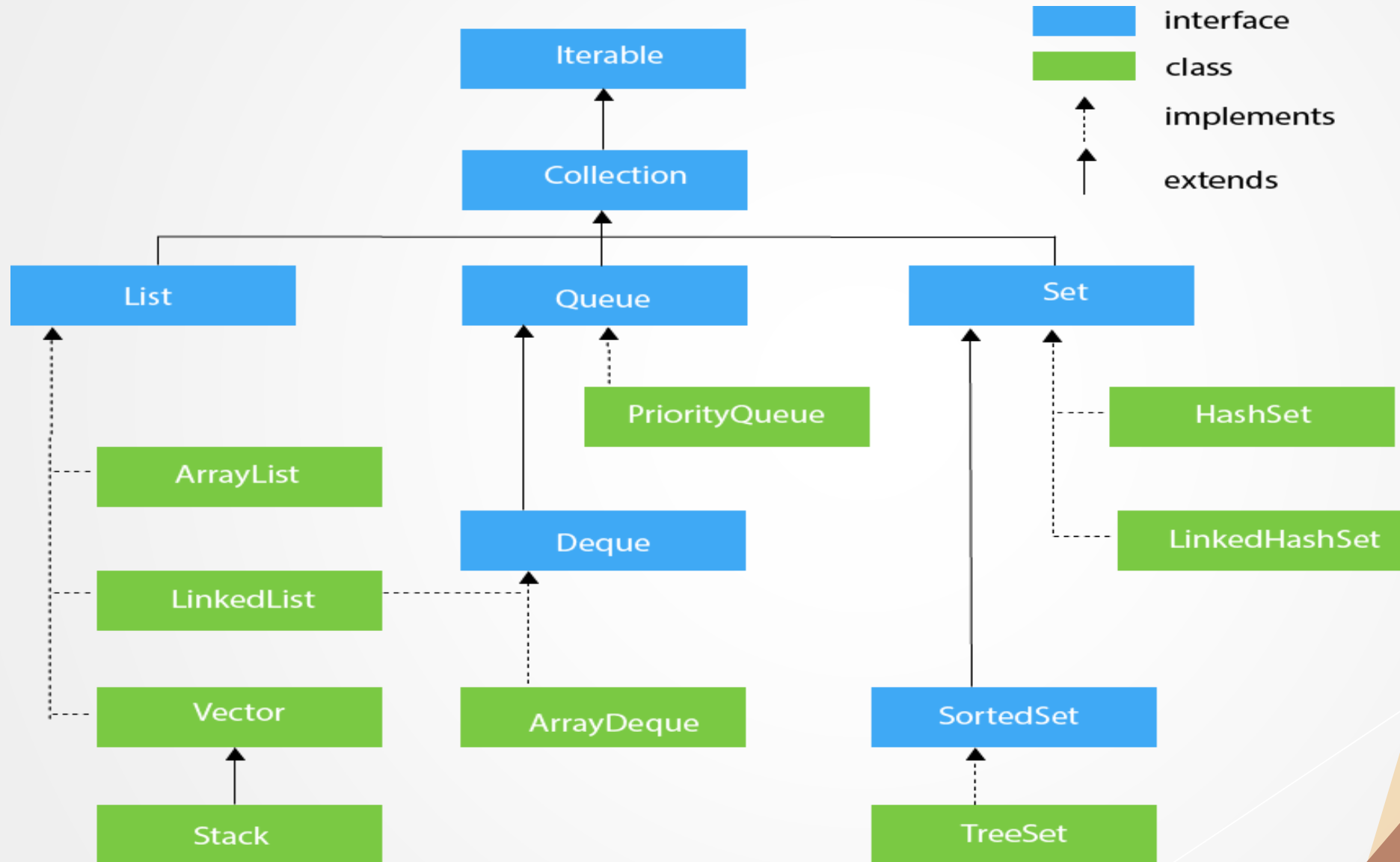


Collection Framework and JDBC

Collection Framework

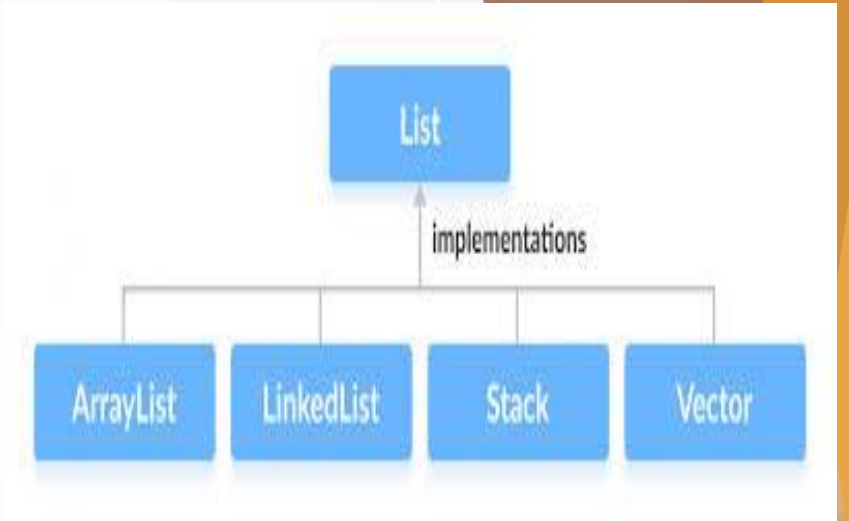
- ▶ The Collection in Java is a framework that provides an architecture to store and manipulate the group of objects.
- ▶ Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.
- ▶ Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (Array List, Vector, LinkedList, PriorityQueue, HashSet, Linked HashSet, Tree Set)
- ▶ To Overcome the limitation of Primitive data types and array we use the collection
- ▶ The collection growable in size i.e. Base on our requirement we can increase or decrease the size.

► Hierarchy of Collection Framework



List interface

- ▶ List interface is the child interface of Collection interface.
- ▶ It represents an ordered collection (also known as a sequence).
- ▶ List allow duplicate Objects
- ▶ Insertion order is preserved
- ▶ The implementation classes of List interface are ArrayList, LinkedList, Stack and Vector.
- ▶ There is three Approach to declare the List interface
 1. By raw type object creation : -
`List obj1 = new ArrayList();`
 2. By with WrapperClasses :-
`List<WrapperClass> Obj2 = new LinkedList();`
 3. Generic type object creation with User defined classes
`List<Student> Obj1=new ArrayList();`



ArrayList

- ▶ Array list is a part of the Collection framework present in java.util package.
- ▶ ArrayList supports dynamic array that can grow as needed.
- ▶ It can contain Duplicate elements and it also maintains the insertion order.
- ▶ Insertion Order is Preserved.
- ▶ Manipulation is slow because a lot of shifting needs to be occurred if any element is removed from the array list.
- ▶ Basic Operations on ArrayList :
 - ▶ Add elements
 - ▶ Access elements
 - ▶ Change elements
 - ▶ Remove elements
- ▶ Array list elements can be accessing throw:
 - ▶ For each loop and
 - ▶ Iterator

Iterator

```
import java.util.*;

public class ArrayListExample2{
    public static void main(String args[]){
        ArrayList<String> list=new ArrayList<String>();
        list.add("Mango");
        list.add("Apple");
        list.add("Banana");
        list.add("Grapes");

        Iterator itr=list.iterator();
        while(itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}
```

For each loop

```
import java.util.*;

public class ArrayListExample3{
    public static void main(String args[]){
        ArrayList<String> list=new ArrayList<String>();
        list.add("Mango");//Adding object in arraylist
        list.add("Apple");
        list.add("Banana");
        list.add("Grapes");
        for(String fruit:list)
            System.out.println(fruit);
    }
}
```

LinkedList

- ▶ Linked List is a part of the Collection framework present in java.util package
- ▶ Insertion order is preserved.
- ▶ Duplicates are allowed.
- ▶ Accept homogenous and heterogeneous elements also..
- ▶ LinkedList implements Serializable and cloneable interfaces but not random Access interface.
- ▶ create Linked List Object
- ▶ List list=new LikedList(); // Raw Type
- ▶ List<Double> price=new LikedList<Double>();
// Generic Type with Wrapper Class
- ▶ List<Movies> movies=new LinkedList<Movies>();
// Generic Type with user defined Classes

```
import java.util.*;
public class GFG {
    public static void main(String args[])
    {
        LinkedList<String> ll = new LinkedList<String>();
        ll.add("A");
        ll.add("B");
        ll.addLast("C");
        ll.addFirst("D");
        ll.add(2, "E");

        System.out.println(ll);

        ll.remove("B");
        ll.remove(3);
        ll.removeFirst();
        ll.removeLast();

        System.out.println(ll);
    }
}
```

```
[D, A, E, B, C]
[A]
```

vector

- ▶ Vector is a class that is implemented in the collection framework implements a growable array of objects
- ▶ Vector implements a dynamic array that means it can grow or shrink as required. Like an array, it contains components that can be accessed using an integer index.
- ▶ Duplicates are allowed.
- ▶ Insertion order preserved.
- ▶ null insertion is possible.
- ▶ homegenious and hetrogenious data elements allowed.
- ▶ Vector class implemented Serializable, cloneable and
- ▶ Most of the methods present in vector are synchronized. hence Vector objects are Thread safe.
- ▶ ex:
- ▶ `Vector<String> names=new Vector<>();`

Stack

- ▶ Stack is a child class of Vector.
- ▶ It is specially designed class for Last In First Out (LIFO).
- ▶ Stack implemented Methods
 - Object push(object o): for inserting an object to the stack
 - Object pop(): To removes and returns top of the stack
 - Object peek(): To Returns top of the stack without removal of Object.
 - int search(object obj): if the specified object is available it return its offset from to of the stack
- ▶ Syntax:
- ▶ Stack s=new Stack();

```
import java.io.*;
import java.util.*;

class GFG {
    public static void main(String[] args)
    {
        int n = 5;
        List<Integer> s = new Stack<Integer>()
        for (int i = 1; i <= n; i++)
            s.add(i);
        System.out.println(s);
        s.remove(3);
        System.out.println(s);
        for (int i = 0; i < s.size(); i++)
            System.out.print(s.get(i) + " ");
    }
}
```

```
[1, 2, 3, 4, 5]
[1, 2, 3, 5]
1 2 3 5
```

SET interface

- ▶ set is a child interface of Collection Interface.
- ▶ If we want to represents a group of indivisual objects as a single entity. where duplicates are not allowed and insertion order is not preserved (random order) then we should go for Set Interface.
- ▶ Purpose of the Set Interface
 - To eliminate duplicate entries from a collection.
 - Ideal for **mathematical set operations**
- ▶ Implemented Classes of Set Interface
 - HashSet
 - LinkedHashSet,
 - TreeSet

HashSet

- ▶ HashSet is a Unordered collection that uses a hash table for storing elements
- ▶ It's the faster set implementation for storing and retrieving Unique elements
- ▶ Duplicate are not allowed .if we are trying to insert duplicate we won't get any compile time error. but it cat print unique values only.
- ▶ Insertion order is not preserved all object will be inserted based on hash-code of objects.
- ▶ Hetrogenious objects are allowed.
- ▶ Implements serializable and cloneable interfaces but not random access.
- ▶ HashSet is the best choice. if our frequent operations are searching.

Example

```
▶ import java.util.*;
▶ public class HashSetExp {
▶     public static void main(String[] args) {
▶         HashSet<Integer> rollNos=new HashSet<>();
▶         rollNos.add(null);
▶         rollNos.add(101);
▶         rollNos.add(101);
▶         rollNos.add(null);
▶         System.out.println(rollNos);
▶     }
▶ }
```

2. LinkedHashSet

- ▶ it is the child class of HashSet
- ▶ The underlying data structure is HashTable+LinkedList
- ▶ It is Exactly same as HashSet except Insertion Order that Means Insertion Order is Preserved. Means Order follows.
- ▶ Maintains **insertion order**.
- ▶ Slightly slower than HashSet due to order maintenance
- ▶ The LinkedHashSet is an ordered version of HashSet that maintains a doubly-linked List across all elements. When the iteration order is needed to be maintained this class is used.

Example

```
▶ import java.util.*;  
▶ public class LinkedHashSetExpAppn {  
▶     public static void main(String[] args) {  
▶         LinkedHashSet<Integer> accountNos=new LinkedHashSet<>();  
▶         accountNos.add(123);  
▶         accountNos.add(456);  
▶         accountNos.add(789);  
▶         accountNos.add(987);  
▶         System.out.println(accountNos);  
▶     }  
▶ }
```

3. TreeSet

- ▶ A TreeSet Stores elements in a sorted order
- ▶ Duplicate Objects not allowed.
- ▶ insertion order not preserved, but all objects will be inserted according to some sorting order.
- ▶ Homogenous elements only accepted.
- ▶ if try to insert hetrogenious we will runtime error.
- ▶ default natural sorting order for numbers Accending order and for string Alphabetical order.
- ▶ Slower Peformance
- ▶ It has the Limited Null Handling

Example

```
import java.util.*;
class TreeSet1{
    public static void main(String args[])
    {
        TreeSet<String> al=new TreeSet<String>();
        al.add("Ravi");
        al.add("Vijay");
        al.add("Ravi");
        al.add("Ajay");

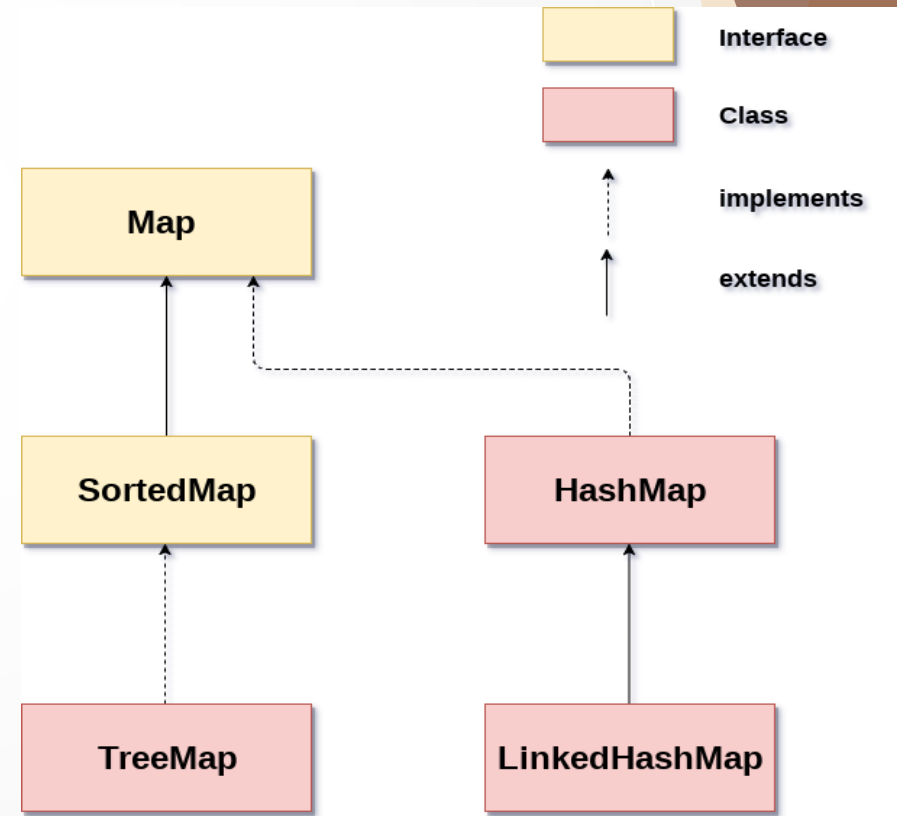
        Iterator<String> itr=al.iterator();
        while(itr.hasNext()){
            System.out.println(itr.next());
        }
    }
}
```

OUTPUT

Ravi
Vijay
Ajay

Map interface

- ▶ The map interface in Java is a structure that holds a set of key-value pairs where each key is unique and points to one value only
- ▶ A Map is useful if you have to search, update or delete elements on the basis of a key.
- ▶ A Map doesn't allow duplicate keys, but you can have duplicate values.
- ▶ Map interface implemented classes
 1. HashMap: here providing map object not in order.
 2. LinkedHashMap: here map follows the insertion order
 3. TreeMap: here map object takes in sorting order.



HashMap

- ▶ Java HashMap class implements the Map interface which allows us to store key and value pair, where keys should be unique.
- ▶ If you try to insert the duplicate key, it will replace the element of the corresponding key.
- ▶ Unordered Collection: Does not guarantee any specific order of keys or values.
- ▶ EX

```
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
        HashMap<String, String> capitalCities = new HashMap<String, String>();
        capitalCities.put("England", "London");
        capitalCities.put("Germany", "Berlin");
        capitalCities.put("Norway", "Oslo");
        capitalCities.put("USA", "Washington DC");
        System.out.println(capitalCities);
        capitalCities.get("England");
        System.out.println(capitalCities.get("England"));
        capitalCities.remove("England");
        System.out.println(capitalCities);
    }
}
```

```
{USA=Washington DC, Norway=Oslo, England=London, Germany=Berlin}
London
{USA=Washington DC, Norway=Oslo, Germany=Berlin}
```

LinkedHashMap

- ▶ The LinkedHashMap Class is just like HashMap with an additional feature of maintaining an order of elements inserted into it.
- ▶ HashMap provided the advantage of quick insertion, search, and deletion but it never maintained the track and order of insertion,
- ▶ which the LinkedHashMap provides where the elements can be accessed in their insertion order.
- ▶ Fast Access
- ▶ Allow Null Values

```
import java.util.*;
class LinkedHashMap2{
    public static void main(String args[]){
        LinkedHashMap<Integer, String> map = new LinkedHashMap<Integer, String>();
        map.put(100,"Amit");
        map.put(101,"Vijay");
        map.put(102,"Rahul");
        //Fetching key
        System.out.println("Keys: "+map.keySet());
        //Fetching value
        System.out.println("Values: "+map.values());
        //Fetching key-value pair
        System.out.println("Key-Value pairs: "+map.entrySet());
    }
}
```

Keys: [100, 101, 102]

Values: [Amit, Vijay, Rahul]

Key-Value pairs: [100=Amit, 101=Vijay, 102=Rahul]

TreeMap

- ▶ The TreeMap in Java is a concrete implementation of the `java.util.SortedMap` interface.
- ▶ It provides an ordered collection of key-value pairs, where the keys are ordered based on their natural order or a custom `Comparator` passed to the constructor.
- ▶ A TreeMap is implemented using a Red-Black tree, which is a type of self-balancing binary search tree.
- ▶ This provides efficient performance for common operations such as adding, removing, and retrieving elements,
- ▶ With an average time complexity of $O(\log n)$.
- ▶ TreeMap in Java does not allow null keys

```
import java.util.Map;
import java.util.TreeMap;

public class Main {
    public static void main(String[] args)
    {
        Map<String, Integer> treeMap = new TreeMap<>();

        // Adding elements to the tree map
        treeMap.put("A", 1); // O(Log n)
        treeMap.put("C", 3); // O(Log n)
        treeMap.put("B", 2); // O(Log n)

        // Getting values from the tree map
        int valueA = treeMap.get("A"); // O(Log n)
        System.out.println("Value of A: " + valueA);

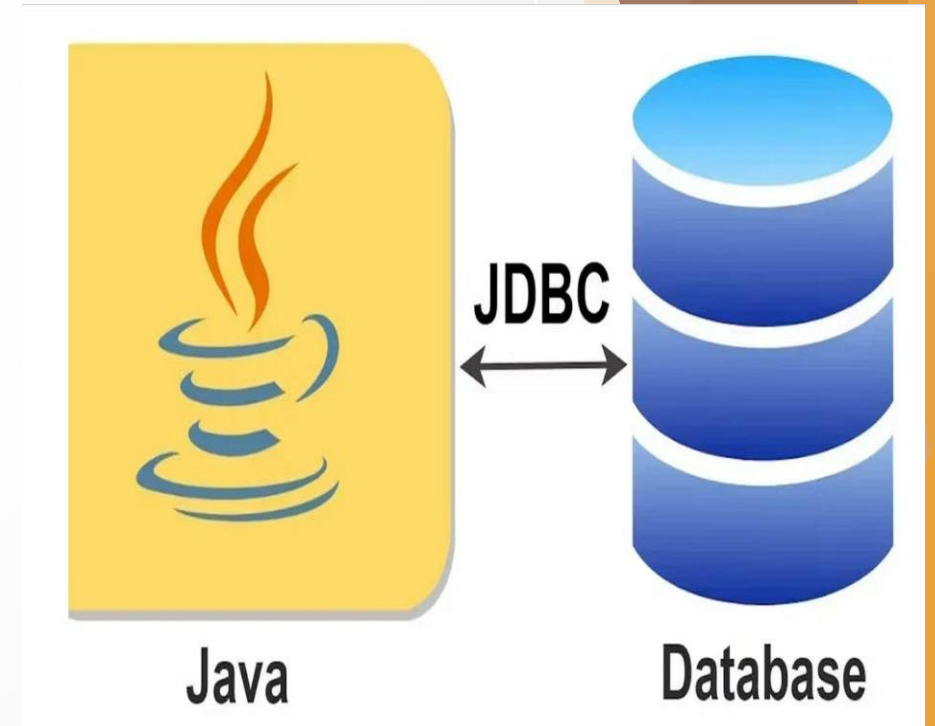
        // Removing elements from the tree map
        treeMap.remove("B"); // O(Log n)

        // Iterating over the elements of the tree map
        for (String key : treeMap.keySet()) { // O(n)
            System.out.println(
                "Key: " + key + ", Value: "
                + treeMap.get(key)); // O(Log n) for each
                                   // get operation
        }
    }
}
```

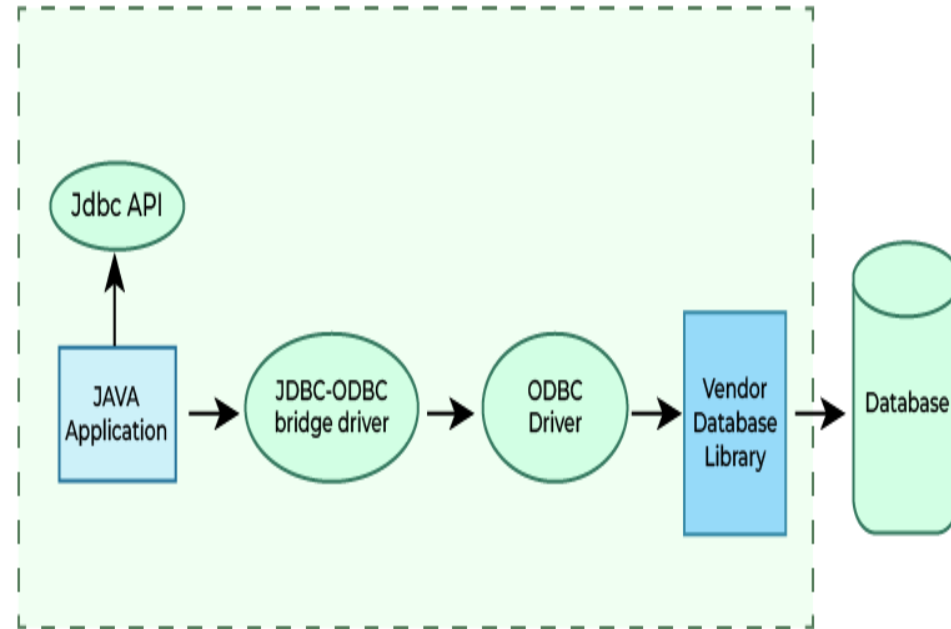
```
Value of A: 1
Key: A, Value: 1
Key: C, Value: 3
```

JDBC

- ▶ JDBC Stands for Java Data Base Connectivity.
- ▶ The purpose of JDBC Is to establishing connection from Java Applications to Any Relational data bases.
- ▶ Java Database Connectivity (JDBC) is an application programming interface (API) for the programming language Java, which defines how a client may access any kind of tabular data, especially a relational database.
- ▶ And Perform CRUD Operations.
- ▶ JDBC is a Java API to connect and execute the query with the database.
- ▶ The use of API's are to enable connection form one applications to another application and interchange the data (information).
- ▶ API's are consist with predefined interfaces, classes and methods .
- ▶ Ex:
 1. JDBC API :
 2. Servlet API: dev loping web application
 3. JSP API: User Interface (UI) and Web Based Application

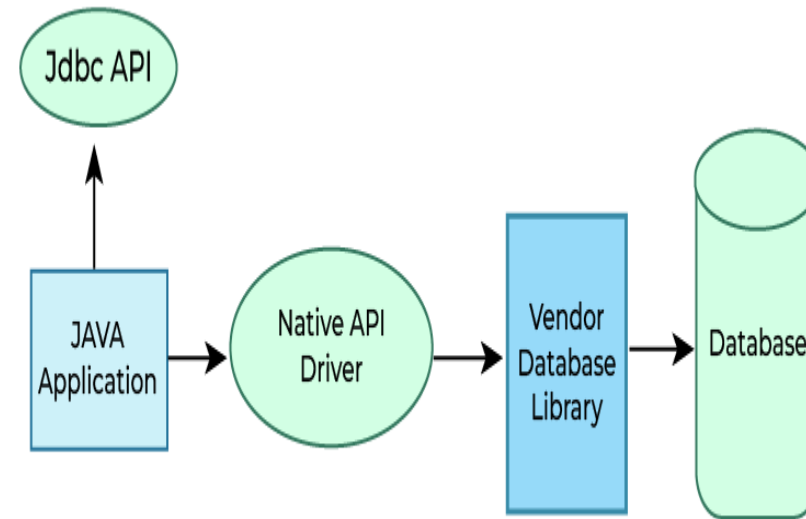


- ▶ JDBC drivers are client-side adapters (installed on the client machine, not on the server) that convert requests from Java programs to a protocol that the DBMS can understand.
- ▶ There are 4 types of JDBC drivers.
- ▶ **1. JDBC-ODBC bridge driver – Type 1 driver**
- ▶ JD JDBC-ODBC bridge driver uses ODBC driver to connect to the database.
- ▶ The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls.
- ▶ Type-1 driver is also called Universal driver because it can be used to connect to any of the databases.
- ▶ It is a database independent driver.



- ▶ **2. Native-API driver – Type 2 driver**

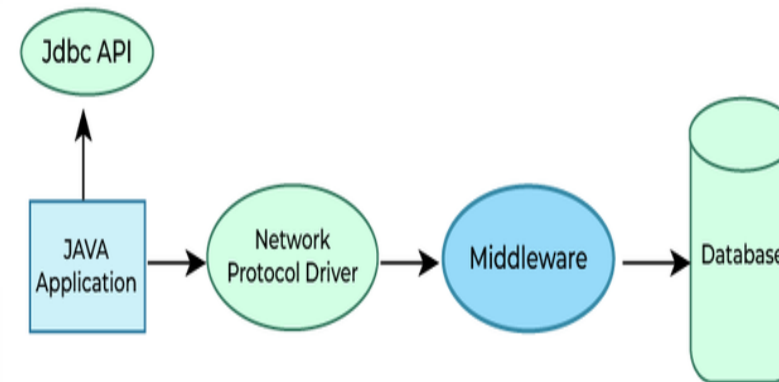
- ▶ The Native API driver uses the client -side libraries of the database.
- ▶ This driver converts JDBC method calls into native calls of the database API
- ▶ In order to interact with different database, this driver needs their local API, that's why data transfer is much more secure as compared to type-1 driver.
- ▶ This driver is not fully written in Java that is why it is also called Partially Java driver.
- ▶ Native-API driver gives better performance than JDBC-ODBC bridge driver.



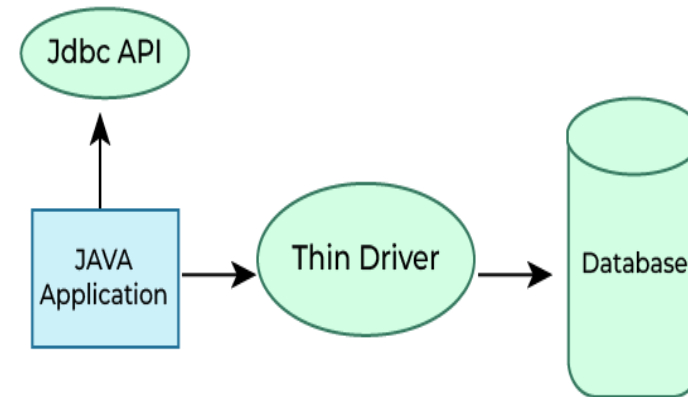
- ▶ **3 .Network Protocol driver – Type 3 driver (fully Java driver)**
- ▶ The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol.
- ▶ Here all the database connectivity drivers are present in a single server, hence no need of individual client-side installation.

Type-3 drivers are fully written in Java, hence they are portable drivers.

- ▶ No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.
- ▶ Network support is required on client machine.



- ▶ **4. Thin driver – Type 4 driver (fully Java driver)**
- ▶ Type-4 driver is also called native protocol driver. This driver interact directly with database. It does not require any native database library, that is why it is also known as Thin Driver.
- ▶ Does not require any native library and Middleware server, so no client-side or server-side installation.
- ▶ It is fully written in Java language, hence they are portable drivers.



- ▶ To enable connection from Java Applications to Data Base JDBC Follows 5 Steps

- ▶ **1. Register The Driver class :**

The forName() method of Class is used to register the driver class. This method is used to dynamically load the driver class.

Statement

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

- ▶ **2. Create a Connection :**

The getConnection() method of DriverManager class is used to establishing connection with Data Base

Statement:

```
Connection con=DriverManager.getConnection(URL,UN>Password);
```

Connection

```
con=DriverManager.getConnection("jdbc:mysql://localhost:3306/dbname","root","root");
```

▶ 3. create statement :

To create sql statement in JDBC 3 ways

1. By Using Statement interface --> for Static Queries
2. By Using PreparedStatement --> for Dynamic Queries
3. By CallableStatement --> execute Stored procedure

1. Statement stmt=con.createStatement();

// DDL commands and static queries

2.PreparedStatement ps = con.prepareStatement(" ");

// DML commands and Dynamic Queries

3.CallableStatement cl = con.prepareCall("stored procedure call");

// storedProcedure

▶ 4. execute SQL queries In Data Base :

Basically execute SQL stament in Data Base we have 2 methods

1. executeUpdate():
2. executeQuery():

1. executeUpdate():

the method is used to execute DML queries in Data Base insert, update and delete.

- ▶ once Query is executed in data Base it can return boolean values i.e 1, 0. Here 1 means query executed Successfully. 0--> means query not executed

- ▶ ex:

```
int l = stmt.executeUpdate("insert into emp values(v1,v2,v3...)");
```

2. executeQuery():

the method is used to execute DRL query in database. Select (display records).

executeQuery() method return ResultSet object.

ex:

```
ResultSet rs = stmt.executeQuery("select *from emp");
```

▶ 5. close The connection :

- ▶ The close() method of connection interface is used to close the connection with data base
con.close();

```
import java.sql.*;
import java.util.Scanner;
public class Inserting {

    public static void main(String[] args) {
        try
        {
            for(int j=0; j<= 5; j++) {
                Class.forName("com.mysql.cj.jdbc.Driver");
                Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/db","root","Madhav@168");
                Scanner sc= new Scanner(System.in);
                System.out.println("Enter the values of customer id =, name ,age");
                int cid=sc.nextInt();
                String cname =sc.next();
                int cage = sc.nextInt();
                System.out.println("Enter email id =");
                String email=sc.next();
                PreparedStatement ps = con.prepareStatement("insert into customer values(?,?,?,?)");
                ps.setInt(1, cid);
                ps.setString(2,cname);
                ps.setInt(3, cage);
                ps.setString(4, email);
                int i=ps.executeUpdate();
                System.out.println("new records inserted successfully");
            }
        }
        catch(Exception ex)
        {
            ex.printStackTrace();
        }
    }
}
```